Overview & Design Philosophy

MedBill Lens is a privacy-first Chrome extension with three independent tools in one package. Every feature runs entirely inside the user's browser. No patient data is ever transmitted to any server. This keeps the product outside HIPAA jurisdiction and keeps infrastructure cost at zero.

CHROME EXTENSION

Principle	Implementation	Why it matters
Stateless by design	No backend, no database	Zero HIPAA exposure
Local-first processing	PDF.js + Regex run in-browser	Works offline, no latency
No DOM scraping	Paste or upload — user controls	Survives portal redesigns
Bundled data	CMS + ICD-10 JSON in extension	Instant lookup, no API cost
Open escape valve	Report Portal button	Handles edge cases gracefully

Tech Stack

Component	Technology	Cost	Notes
Extension shell	Manifest V3 + Vanilla JS	Free	No React — keeps bundle tiny
PDF parsing	PDF.js (Mozilla)	Free	Runs 100% in browser
Code search	Fuse.js fuzzy search	Free	Offline capable
Price data	RxNav API (US Govt)	Free	No key needed
Drug pricing	Cost Plus Drugs CSV	Free	Mark Cuban's pharmacy
Landing page	Next.js on Vercel	Free	Static export, no server
Domain	Porkbun / Namecheap	~$9/yr	medbilllens.com
Chrome store	Google Developer account	$5 one-time	Lifetime, not recurring

Total cost to ship: ~$14 (domain $9 + Chrome store $5). All other components are free forever.

Comp

Every file in the extension, its purpose, and approximate size:

```
medbill-lens/
■■■ manifest.json              ← Extension config, permissions, tab routing
■■■ popup/
■   ■■■ popup.html             ← 3-tab shell UI (Tab 1 / 2 / 3 navigation)
■   ■■■ popup.css              ← Styles: tab bar, cards, color coding
■   ■■■ popup.js               ← Tab switcher, coordinates all three modules
■■■ tabs/
■   ■■■ medbill/
■   ■   ■■■ medbill.js         ← CPT extractor, price lookup, overcharge flag
■   ■   ■■■ medbill.css        ← Bill result cards, green/red highlights
■   ■■■ codesearch/
■   ■   ■■■ codesearch.js      ← Fuse.js search engine, result renderer
■   ■   ■■■ codesearch.css     ← Search input, result list styling
■   ■■■ rxscout/
■       ■■■ rxscout.js         ← RxNav API caller, Cost Plus price lookup
■       ■■■ rxscout.css        ← Drug result cards, disclaimer styling
■■■ data/
■   ■■■ cpt_medicare_prices.json   ← CMS fee schedule (bundled, ~4MB)
■   ■■■ icd10_cpt_codes.json       ← ICD-10 + modifiers (bundled, ~8MB)
■■■ lib/
■   ■■■ fuse.min.js            ← Fuzzy search library (offline)
■   ■■■ pdf.min.js             ← PDF.js (Mozilla, offline)
■■■ icons/
■   ■■■ icon16.png
■   ■■■ icon48.png
■   ■■■ icon128.png
■■■ background.js              ← Minimal service worker (Manifest V3 req.)
```

manifest.json — key fields

```
{
  "manifest_version": 3,
  "name": "MedBill Lens",
  "version": "1.0.0",
  "description": "Audit medical bills, search codes, compare drug prices.",
  "permissions": [],
  "host_permissions": [
    "https://rxnav.nlm.nih.gov/*"
  ],
  "action": {
    "default_popup": "popup/popup.html",
    "default_icon": { "48": "icons/icon48.png" }
  },
  "background": {
    "service_worker": "background.js"
  },
  "content_security_policy": {
    "extension_pages": "script-src 'self'; object-src 'self'"
  }
}
```

No content_scripts needed — zero page access. Only RxNav API call requires host_permissions.

TAB 1 — MedBill Lens — Bill Auditor — Reliability: Medium

What it does

User provides their medical bill via paste or PDF upload. The extension extracts all CPT procedure codes using regex, looks each one up against the bundled CMS Medicare fee schedule, and displays: the procedure name, what Medicare pays, what the user was charged, and a green (fair) or red (overcharged) flag. Premium users get a one-click PDF dispute letter.

User flow — two input modes

Step	Mode A: Paste	Mode B: PDF Upload
1	Open bill in MyChart / portal	Download PDF from portal
2	Ctrl+A, Ctrl+C to copy all text	Click upload area in extension
3	Paste into extension text box	PDF.js extracts text locally
4	Regex scans for CPT codes	Regex scans for CPT codes
5	JSON lookup → Medicare price	JSON lookup → Medicare price
6	Results displayed with color flags	Results displayed with color flags
7 (Premium)	Download PDF dispute letter ($9)	Download PDF dispute letter ($9)

Core logic — medbill.js

```
// CPT code regex (5-digit codes, optionally followed by modifier)
const CPT_REGEX = /\b(\d{5})([-][A-Z0-9]{2})?\b/g;

function extractCPTCodes(text) {
  const matches = [];
  let match;
  while ((match = CPT_REGEX.exec(text)) !== null) {
    matches.push({ code: match[1], modifier: match[2] || null });
  }
  return [...new Map(matches.map(m => [m.code, m])).values()]; // dedupe
}

async function lookupPrices(codes) {
  // cpt_medicare_prices.json is loaded once at extension startup
  const results = codes.map(({ code }) => {
    const entry = CPT_DATA[code];
    if (!entry) return { code, found: false };
    return {
      code,
      description:    entry.description,
      medicare_price: entry.non_facility_price,
      found: true
    };
  });
  return results;
}

function flagOvercharge(charged, medicare) {
  const ratio = charged / medicare;
  if (ratio > 3.0) return { flag: 'EXTREME',  color: '#A32D2D' };
  if (ratio > 2.0) return { flag: 'HIGH',     color: '#D85A30' };
  if (ratio > 1.5) return { flag: 'MODERATE', color: '#BA7517' };
  return           { flag: 'FAIR',     color: '#3B6D11' };
}
```

Data source: CMS Medicare Physician Fee Schedule

The CMS publishes the Medicare Physician Fee Schedule annually as a free download. It contains non-facility and facility prices for every CPT code. You bundle this as a JSON file inside the extension — no API call, no cost, works offline. Update annually when CMS releases the new year's file (typically November).

Download: cms.gov/medicare/physician-fee-schedule/search

Monetization — Premium dispute letter

Tier	Features	Price
Free	CPT extraction, Medicare price comparison, color flags	$0
Premium	All free features + downloadable PDF dispute letter personalized with their specific CPT codes and overcharges	$9 one-time

TAB 2 Medical Code Search — ICD-10 / CPT / Modifiers Reliability: Very High

What it does

A fast, keyboard-driven search engine for medical codes. User types any part of a code or description and gets instant fuzzy-matched results from the bundled database. Covers ICD-10 diagnosis codes, CPT procedure codes, and modifier codes. This tab has zero external dependencies — it works offline and has no failure modes.

Why this tab builds trust

Patients researching their bills will look up the codes they see. Having a fast, clean lookup tool inside the same extension creates a flywheel: they come for the code search, they stay for the bill audit. It also serves billers and medical students, broadening the addressable audience without adding complexity.

Core logic — codesearch.js

```
import Fuse from '../lib/fuse.min.js';

// Fuse.js config — optimized for medical code search
const FUSE_OPTIONS = {
  keys: [
    { name: 'code',        weight: 0.4 },
    { name: 'description', weight: 0.6 },
  ],
  threshold: 0.35,      // 0 = exact, 1 = match anything
  minMatchCharLength: 2,
  includeScore: true,
  includeMatches: true, // for highlight
};

let fuse = null;

async function initSearch() {
  // icd10_cpt_codes.json loaded once at startup
  const data = await loadJSON('../data/icd10_cpt_codes.json');
  fuse = new Fuse(data, FUSE_OPTIONS);
}

function search(query) {
  if (!fuse || query.length < 2) return [];
  return fuse.search(query, { limit: 20 }).map(r => ({
    code:        r.item.code,
    description: r.item.description,
    type:        r.item.type,    // 'ICD10' | 'CPT' | 'MODIFIER'
    score:       r.score,
    matches:     r.matches,      // for bold highlighting
  }));
}
```

Data sources

Dataset	Source	Records	Update freq.
ICD-10-CM codes	CDC / CMS (cms.gov)	~72,000	Annual (Oct)
CPT codes	AMA (public subset via CMS)	~10,000	Annual (Jan)
Modifier codes	CMS Physician Fee Schedule	~150	Annual

HCPCS Level II	CMS (cms.gov/medicare/coding)	~7,000	Quarterly

All datasets are published by US government agencies and are free to use. Bundle them as a single minified JSON at build time to keep the extension size manageable. Total bundled size after minification: approximately 6-8 MB.

JSON data format

```
{
  "code": "J45.51",
  "description": "Severe persistent asthma with acute exacerbation",
  "type": "ICD10",
  "category": "Respiratory"
}

{
  "code": "99213",
  "description": "Office visit, established patient, low complexity",
  "type": "CPT",
  "category": "Evaluation & Management"
}
```

TAB 3 — Rx Price Scout — Prescription Price Comparison

Reliability: Medium-High

What it does

User types a drug name. The extension calls the RxNav API (US National Library of Medicine) to get the drug's RxNorm ID, retrieve all available formulations, and identify generic equivalents. It then cross-references against the Mark Cuban Cost Plus Drugs pricing CSV to show reference prices. A clear disclaimer is always shown: prices are reference only and vary by location.

Why GoodRx was avoided

GoodRx shut down public API access. Scraping their site violates their ToS and would get the extension blocked. RxNav + Cost Plus Drugs is the correct free, sustainable alternative that will not disappear.

Core logic — rxscout.js

```
const RXNAV_BASE = 'https://rxnav.nlm.nih.gov/REST';

async function searchDrug(name) {
  // Step 1: Get RxNorm concept ID
  const approx = await fetch(
    `${RXNAV_BASE}/approximateTerm?term=${encodeURIComponent(name)}&maxEntries=5`
  ).then(r => r.json());

  const candidates = approx.approximateGroup?.candidate || [];
  if (!candidates.length) return { found: false };

  const rxcui = candidates[0].rxcui;

  // Step 2: Get drug details and generic alternatives
  const [details, generics] = await Promise.all([
    fetch(`${RXNAV_BASE}/rxcui/${rxcui}/property?propName=RxNorm+Name`).then(r => r.json()),
    fetch(`${RXNAV_BASE}/rxcui/${rxcui}/related?tty=SBD+SCD`).then(r => r.json()),
  ]);

  // Step 3: Cross-reference Cost Plus Drugs CSV (pre-loaded at startup)
  const costPlusPrice = COST_PLUS_DATA[rxcui] || lookupByName(name);

  return {
    rxcui,
    name:          details.propConceptGroup?.propConcept?.[0]?.propValue || name,
    generics:      extractGenerics(generics),
    costPlusPrice, // null if not in Cost Plus catalogue
    disclaimer:    'Reference price only. Actual cost varies by pharmacy, insurance, and location.',
  };
}
```

Data sources

Source	What it provides	Cost	API key?
RxNav (NLM)	Drug name → RxNorm ID, formulations, generics	Free	No
Cost Plus Drugs	Generic drug prices from Mark Cuban's pharmacy	Free CSV	No

Cost Plus CSV download: costplusdrugs.com/drugs/list — updated regularly. Bundle a snapshot at build time, supplement with live RxNav calls.

Build Order & Reliability Matrix

Recommended build sequence — start with zero-risk, ship early, add complexity:

Phase	What to build	Time est.	Dependency risk
1	Tab 2 — Code Search Fuse.js + bundled JSON. No API calls. Works offline from day one.	2-3 days	None
2	Extension shell — popup.html + tab switcher The 3-tab UI skeleton that hosts all three tools.	1 day	None

3	Tab 1 — MedBill Lens (paste mode) Text area + CPT regex + CMS JSON lookup. No PDF yet.	2-3 days	Low
4	Tab 1 — PDF upload mode Add PDF.js drag-and-drop. Reuses same parser from step 3.	1-2 days	Low
5	Tab 3 — Rx Price Scout RxNav API integration + Cost Plus CSV cross-reference.	2-3 days	Medium
6	Report Portal button Captures page URL + timestamp, writes to Supabase free tier.	1 day	Low
7	Premium dispute letter PDF generation using jsPDF (browser-side). Gated behind $9 payment.	2-3 days	Low

Reliability matrix

Tab	Data reliability	Scraping risk	API dependency	Overall
MedBill Lens	High — CMS annual file	None — paste/upload	None	Medium*
Code Search	Very high — static bundle	None	None	Very High
Rx Scout	Medium — ref prices only	None	RxNav (govt, stable)	Medium-High

** MedBill Lens data is reliable. 'Medium' overall reflects variability in how cleanly different portals export bill text — not a data quality issue.*

Popup UI — HTML Structure

The extension popup is a single HTML file with three tab panels. Width: 420px. Height: auto (max 580px). No external CSS framework.

```
<!-- popup/popup.html -->
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="popup.css">
</head>
<body>
  <header class="app-header">
    <img src="../icons/icon16.png" alt=""/>
    <span class="app-name">MedBill Lens</span>
    <span class="version">v1.0</span>
  </header>

  <!-- Tab navigation -->
  <nav class="tab-bar">
    <button class="tab active" data-tab="medbill">
      Bill Audit
    </button>
    <button class="tab" data-tab="codesearch">
      Code Search
    </button>
    <button class="tab" data-tab="rxscout">
      Rx Prices
    </button>
  </nav>

  <!-- Tab panels -->
  <main>
    <section id="tab-medbill" class="tab-panel active">
      <!-- paste textarea + PDF drop zone + results -->
    </section>

    <section id="tab-codesearch" class="tab-panel">
      <!-- search input + result list -->
    </section>

    <section id="tab-rxscout" class="tab-panel">
      <!-- drug name input + price results + disclaimer -->
    </section>
  </main>

  <footer class="app-footer">
    <!-- "Report portal issue" button (worst-case escape valve) -->
    <button id="report-portal">Report portal issue</button>
  </footer>

  <script src="popup.js"></script>
</body>
</html>
```

Report Portal button — escape valve logic

When the user's portal layout prevents clean copy-paste, this button captures: the current page URL, the browser/OS, and the portal name if detectable. It writes a single row to a Supabase free-tier table. You review weekly and write a targeted content script for the most-reported portal. No PHI — just URL and browser metadata.

```
// report-portal button handler
document.getElementById('report-portal').addEventListener('click', async () => {
  const [tab] = await chrome.tabs.query({ active: true, currentWindow: true });
  const domain = new URL(tab.url).hostname;  // e.g. "mychart.example.org"

  await fetch('https://YOUR_PROJECT.supabase.co/rest/v1/portal_reports', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'apikey': SUPABASE_ANON_KEY },
    body: JSON.stringify({
      domain,
      browser:    navigator.userAgent,
      reported_at: new Date().toISOString(),
      // NO page content, NO bill data, NO PHI
    })
  });

  showConfirmation('Thanks! We will add support for this portal.');
});
```

Chrome Web Store Submission & Launch

Submission checklist

manifest.json — Version 3, correct permissions (only host_permissions for rxnav.nlm.nih.gov)

Privacy policy — One-page HTML hosted on your domain. Required for medical tools.

Store listing — Screenshots of all 3 tabs. 1280x800 or 640x400.

Description — Lead with 'No data stored. No servers. Runs locally.' — addresses top concern.

Category — Productivity (not Healthcare — smaller category = less competition)

Icon set — 16, 48, 128px PNG. Clean, simple design.

Review time — Budget 3-7 days for first review. Subsequent updates are 1-2 days.

Landing page — what it must say

Section	Content
Hero headline	'Your medical bill, explained in 30 seconds'
Trust line	'No accounts. No servers. Everything stays in your browser.'
Three feature cards	One per tab — Bill Audit, Code Search, Rx Prices
Privacy section	Explicit: zero PHI stored, no analytics, open source option
CTA	'Add to Chrome — Free' button linking to Chrome Web Store
Premium upsell	'Download your dispute letter for $9' — below the fold

Total cost summary

Item	Provider	Cost	Recurring?

Domain	Porkbun / Namecheap	~$9	Annual
Hosting (landing page)	Vercel	Free	No
Chrome Dev account	Google	$5	One-time lifetime
All data sources	CMS, NLM, Cost Plus	Free	No
Portal report storage	Supabase free tier	Free	No (500MB limit)
TOTAL		**~$14**	**$9/yr after year 1**